

Church Management Platform — Complete Initial Implementation Plan

1. Project Objective

Develop a scalable and customizable church management platform to centralize administrative, organizational, and ministerial information and activities.

The system will initially be developed and configured for **one specific church**. It will not support the registration or management of multiple churches within the same application instance.

If another church requires the platform, the existing codebase will be copied and independently configured with:

- A separate database
- A separate backend API
- A separate deployment environment
- Church-specific settings
- Customized functionalities, workflows, or interface elements

This approach will allow the system to be adapted to the needs of each church without affecting the data, configuration, or operation of other church installations.

The platform will initially include the following modules:

1. Church Management
2. Member Management
3. Ministry Management
4. Announcement Management
5. Event Management
6. Inventory Management
7. Contribution Management
8. Bible Studies Management
9. Document Management

The architecture should remain modular so that new functionalities and modules can be implemented in future versions.

2. General System Characteristics

2.1 Single-Church Architecture

Each deployment of the system will serve one church.

The church will not be treated as a tenant or as a record that users can register through the platform. Instead, the church's information will be configured during the system setup.

The system will maintain information such as:

- Church name
- Church logo
- Address
- Contact information
- Social media information
- Church description
- General settings

The database will not require a `churchId` field in every entity because all records in an installation belong to the same church.

2.2 Customization for Different Churches

When deploying the platform for another church:

1. Copy the existing codebase.
2. Create a new database.
3. Configure a separate backend API.
4. Configure a separate frontend environment.
5. Add the church's information and branding.
6. Customize modules or business rules when necessary.
7. Deploy the new version independently.

Changes made for one church should not affect another church's installation.

2.3 Default Language

The default language of the system will be **Portuguese (Brazil)**.

The interface should initially use Portuguese for:

- Navigation
- Buttons
- Forms
- Validation messages
- Notifications
- System messages
- User documentation

The application should be structured to support internationalization in the future. Text displayed in the interface should therefore be organized in translation files rather than being permanently distributed throughout the application code.

Initial language:

Português (Brasil) — pt-BR

Possible future languages:

English — en

Spanish — es

3. Module Implementation Plan

3.1 Church Management

Purpose

Manage the church's general information, branding, settings, users, and administrative configuration.

Basic and Important Features

- Configure church information
- Manage church name and logo
- Manage address and contact information

- Manage church description
- Configure general system settings
- Manage administrative users
- Manage user roles and permissions
- View a general administrative dashboard

Main Information

- Church name
- Logo
- Address
- Telephone number
- Email address
- Social media information
- Description

Main Entities

- Church Settings
- User
- Role
- Permission

Implementation Notes

This module should be implemented first because it establishes the general configuration and administrative structure of the platform.

3.2 Member Management

Purpose

Maintain organized and updated information about church members.

Basic and Important Features

- Register members
- View member profiles
- Edit member information
- Activate or deactivate members
- Search and filter members
- Associate members with ministries

- View basic membership information
- Maintain member history

Main Information

- Full name
- Date of birth
- Contact information
- Address
- Membership status
- Date of membership
- Ministries
- User account, when applicable

Main Entities

- Member
- User
- Ministry Member

Dependencies

- Authentication
 - Authorization
 - Church Management
-

3.3 Ministry Management

Purpose

Organize church ministries, leaders, and participating members.

Basic and Important Features

- Create ministries
- Edit ministry information
- Define ministry descriptions
- Assign ministry leaders
- Add members to ministries
- Remove members from ministries
- View ministry members
- Activate or deactivate ministries

Main Entities

- Ministry
- Ministry Leader
- Ministry Member
- Member

Dependencies

- Member Management
 - Authentication
 - Authorization
-

3.4 Announcement Management

Purpose

Create, organize, and publish announcements to specific groups within the church.

Basic and Important Features

- Create announcements
- Edit announcements
- Save announcements as drafts
- Publish announcements
- Archive announcements
- Define publication dates
- Add images
- Define announcement destinations
- View active and archived announcements

Announcement Destinations

Announcements should support one or more destinations, including:

- All members — default destination
- Visitors
- A specific ministry
- Multiple ministries
- Church leaders
- Specific groups or categories
- All users, when necessary

Example

An administrator may create an announcement for a youth activity and define the destination as:

Ministry: Youth Ministry

Only users associated with the selected ministry should receive or view the announcement.

Main Entities

- Announcement
- Announcement Destination
- Ministry
- Member
- Visitor

Dependencies

- Member Management
 - Ministry Management
 - Authentication
 - Authorization
-

3.5 Event Management

Purpose

Plan, organize, and manage church events.

Basic and Important Features

- Create events
- Edit event information
- Define event dates and times
- Define event locations
- Publish or cancel events
- Register event participants
- View participant lists

- Associate announcements with events
- Display events in a calendar

Main Information

- Event title
- Description
- Start date and time
- End date and time
- Location
- Registration information
- Event status

Main Entities

- Event
- Event Participant
- Member
- Announcement

Dependencies

- Member Management
 - Announcement Management
-

3.6 Inventory Management

Purpose

Manage church-owned items, materials, and supplies.

Basic and Important Features

- Register inventory items
- Define item categories
- Define storage locations
- Record available quantities
- Update item information
- Register stock entries
- Register stock removals
- View inventory movement history
- Identify low-stock items

Main Information

- Item name
- Description
- Category
- Quantity
- Minimum quantity
- Storage location
- Item condition

Main Entities

- Inventory Item
- Inventory Category
- Inventory Movement
- Storage Location

Dependencies

- Authentication
 - Authorization
-

3.7 Contribution Management

Purpose

Manage different types of church contributions, including financial contributions and contributions involving the provision of specific items.

The system should support different contribution models because church activities may require different forms of participation.

Contribution Type 1 — Financial Contributions

This type applies to:

- Offerings
- Donations
- Fundraising campaigns
- Church projects

- Financial support initiatives

Basic Features

- Create a financial contribution campaign
- Define the contribution title
- Define the contribution description
- Define a financial target
- Define the start and end dates
- Allow users to register contributions
- Record contribution amounts
- View contribution history
- Calculate the total amount collected
- Display progress toward the financial target
- Close or archive campaigns

Example

Campaign:
Church Building Renovation

Financial Target:
R\$ 20,000.00

Amount Collected:
R\$ 12,500.00

Progress:
62.5%

Main Entities

- Contribution Campaign
- Financial Contribution
- Member

Contribution Type 2 — Item-Based Contributions

This type applies to events or activities in which specific items are required.

Examples:

- Church dinners
- Conferences

- Retreats
- Community activities
- Social events
- Church celebrations

Administrators should define the items required for the activity. Members should be able to select or offer the items they intend to provide.

Basic Features

- Create an item-based contribution campaign
- Associate the campaign with an event
- Define the required items
- Define the quantity required for each item
- Define the maximum number of users who may offer an item
- Allow users to select items they intend to provide
- Prevent contributions beyond the defined limits
- Display the quantity already offered
- Display the quantity still required
- Allow administrators to approve, update, or cancel commitments
- Close or archive the campaign

Example

Event:
Church Dinner

Required Item:
Soft Drinks

Required Quantity:
30 bottles

Maximum Number of Contributors:
10 members

Current Commitments:
8 members

Quantity Offered:
24 bottles

Quantity Remaining:
6 bottles

Contribution Controls

The system should support controls such as:

- Required quantity
- Quantity already committed
- Quantity remaining
- Maximum number of contributors
- Number of contributors already registered
- Contribution deadline
- Contribution status
- Availability of the item

The system should prevent users from committing to an item when:

- The required quantity has already been reached.
- The maximum number of contributors has been reached.
- The contribution campaign has ended.
- The campaign has been closed.

Main Entities

- Contribution Campaign
- Contribution Type
- Required Item
- Item Commitment
- Member
- Event

Dependencies

- Event Management
- Member Management
- Authentication
- Authorization

3.8 Bible Studies Management

Purpose

Organize Bible studies, study groups, participants, schedules, and learning materials.

Basic and Important Features

- Create Bible studies
- Define study titles and topics

- Define study descriptions
- Assign study leaders
- Define schedules
- Add participants (by ministry or by member)
- Remove participants
- Record attendance
- Associate study materials
- View participant information

Main Entities

- Bible Study
- Bible Study Leader
- Bible Study Participant
- Bible Study Attendance
- Study Material

Dependencies

- Member Management
 - Document Management
-

3.9 Document Management

Purpose

Store, organize, and provide controlled access to church documents.

Basic and Important Features

- Upload documents
- Define document titles
- Define document descriptions
- Categorize documents
- View documents
- Download documents
- Search and filter documents
- Define access permissions
- Associate documents with ministries
- Associate documents with events
- Associate documents with Bible studies
- Archive or delete documents

Main Entities

- Document
- Document Category
- Document Permission
- User

Dependencies

- Authentication
 - Authorization
 - File storage infrastructure
-

4. Functional Requirements

4.1 Church Management

- **FR-CH-01:** The system shall allow authorized administrators to configure the church's general information.
 - **FR-CH-02:** The system shall store the church name, logo, address, contact information, social media information, and description.
 - **FR-CH-03:** The system shall allow administrators to update church information.
 - **FR-CH-04:** The system shall allow administrators to configure general system settings.
 - **FR-CH-05:** The system shall allow administrators to manage platform users.
 - **FR-CH-06:** The system shall allow administrators to assign roles and permissions to users.
 - **FR-CH-07:** The system shall display a general administrative dashboard.
-

4.2 Member Management

- **FR-ME-01:** The system shall allow authorized users to register church members.
- **FR-ME-02:** The system shall allow authorized users to view member profiles.
- **FR-ME-03:** The system shall allow authorized users to update member information.
- **FR-ME-04:** The system shall allow authorized users to activate or deactivate member records.
- **FR-ME-05:** The system shall allow users to search and filter members.

- **FR-ME-06:** The system shall allow members to be associated with one or more ministries.
 - **FR-ME-07:** The system shall maintain member information for historical purposes.
 - **FR-ME-08:** The system shall associate a member with a user account when the member receives platform access.
-

4.3 Ministry Management

- **FR-MI-01:** The system shall allow authorized users to create ministries.
 - **FR-MI-02:** The system shall allow authorized users to update ministry information.
 - **FR-MI-03:** The system shall allow administrators to assign one or more ministry leaders.
 - **FR-MI-04:** The system shall allow members to be added to ministries.
 - **FR-MI-05:** The system shall allow authorized users to remove members from ministries.
 - **FR-MI-06:** The system shall display the members associated with each ministry.
 - **FR-MI-07:** The system shall allow ministries to be activated or deactivated.
-

4.4 Announcement Management

- **FR-AN-01:** The system shall allow authorized users to create announcements.
- **FR-AN-02:** The system shall allow users to save announcements as drafts.
- **FR-AN-03:** The system shall allow authorized users to edit announcements.
- **FR-AN-04:** The system shall allow authorized users to publish announcements.
- **FR-AN-05:** The system shall allow authorized users to archive announcements.
- **FR-AN-06:** The system shall allow announcements to contain titles, descriptions, images, and publication dates.
- **FR-AN-07:** The system shall allow administrators to define one or more announcement destinations.
- **FR-AN-08:** The default announcement destination shall be church members.
- **FR-AN-09:** The system shall allow announcements to be directed to visitors.
- **FR-AN-10:** The system shall allow announcements to be directed to one or more ministries.
- **FR-AN-11:** The system shall display announcements according to their destinations and user permissions.
- **FR-AN-12:** The system shall allow announcements to be associated with events.

4.5 Event Management

- **FR-EV-01:** The system shall allow authorized users to create events.
 - **FR-EV-02:** The system shall allow authorized users to update event information.
 - **FR-EV-03:** The system shall store event titles, descriptions, dates, times, and locations.
 - **FR-EV-04:** The system shall allow authorized users to publish or cancel events.
 - **FR-EV-05:** The system shall allow members to register for events.
 - **FR-EV-06:** The system shall allow authorized users to view event participants.
 - **FR-EV-07:** The system shall display events in a calendar.
 - **FR-EV-08:** The system shall allow announcements to be associated with events.
-

4.6 Inventory Management

- **FR-IN-01:** The system shall allow authorized users to register inventory items.
 - **FR-IN-02:** The system shall allow users to define inventory categories.
 - **FR-IN-03:** The system shall allow users to define storage locations.
 - **FR-IN-04:** The system shall store item quantities and conditions.
 - **FR-IN-05:** The system shall allow authorized users to register stock entries.
 - **FR-IN-06:** The system shall allow authorized users to register stock removals.
 - **FR-IN-07:** The system shall maintain inventory movement history.
 - **FR-IN-08:** The system shall identify items with quantities below the defined minimum.
 - **FR-IN-09:** The system shall allow users to search and filter inventory items.
-

4.7 Contribution Management

General Requirements

- **FR-CO-01:** The system shall allow authorized users to create contribution campaigns.
- **FR-CO-02:** The system shall allow administrators to define the contribution type.
- **FR-CO-03:** The system shall support financial contribution campaigns.
- **FR-CO-04:** The system shall support item-based contribution campaigns.
- **FR-CO-05:** The system shall allow contribution campaigns to be associated with events.

- **FR-CO-06:** The system shall allow administrators to define contribution start and end dates.
- **FR-CO-07:** The system shall allow administrators to activate, close, or archive contribution campaigns.

Financial Contributions

- **FR-CO-08:** The system shall allow administrators to define a financial contribution target.
- **FR-CO-09:** The system shall allow users to register financial contributions.
- **FR-CO-10:** The system shall store the amount and date of each contribution.
- **FR-CO-11:** The system shall calculate the total amount collected.
- **FR-CO-12:** The system shall display progress toward the financial target.
- **FR-CO-13:** The system shall maintain a financial contribution history.

Item-Based Contributions

- **FR-CO-14:** The system shall allow administrators to define the items required for a contribution campaign.
- **FR-CO-15:** The system shall allow administrators to define the required quantity for each item.
- **FR-CO-16:** The system shall allow administrators to define the maximum number of users who may offer each item.
- **FR-CO-17:** The system shall allow users to select items they intend to provide.
- **FR-CO-18:** The system shall allow users to define the quantity they intend to provide.
- **FR-CO-19:** The system shall calculate the quantity already committed for each item.
- **FR-CO-20:** The system shall display the remaining quantity required.
- **FR-CO-21:** The system shall display the number of users who have committed to each item.
- **FR-CO-22:** The system shall prevent commitments that exceed the required quantity.
- **FR-CO-23:** The system shall prevent commitments that exceed the maximum number of contributors.
- **FR-CO-24:** The system shall prevent contributions after the campaign deadline.
- **FR-CO-25:** The system shall allow administrators to manage or cancel item commitments.

4.8 Bible Studies Management

- **FR-BS-01:** The system shall allow authorized users to create Bible studies.
- **FR-BS-02:** The system shall store study titles, descriptions, topics, and schedules.

- **FR-BS-03:** The system shall allow administrators to assign Bible study leaders.
 - **FR-BS-04:** The system shall allow members to be added to Bible studies.
 - **FR-BS-05:** The system shall allow authorized users to remove participants.
 - **FR-BS-06:** The system shall allow authorized users to record participant attendance.
 - **FR-BS-07:** The system shall allow study materials to be associated with Bible studies.
 - **FR-BS-08:** The system shall display participant and attendance information.
-

4.9 Document Management

- **FR-DO-01:** The system shall allow authorized users to upload documents.
 - **FR-DO-02:** The system shall store document titles, descriptions, categories, and upload information.
 - **FR-DO-03:** The system shall allow authorized users to view documents.
 - **FR-DO-04:** The system shall allow authorized users to download documents.
 - **FR-DO-05:** The system shall allow users to search and filter documents.
 - **FR-DO-06:** The system shall allow administrators to define document access permissions.
 - **FR-DO-07:** The system shall allow documents to be associated with ministries.
 - **FR-DO-08:** The system shall allow documents to be associated with events.
 - **FR-DO-09:** The system shall allow documents to be associated with Bible studies.
 - **FR-DO-10:** The system shall allow authorized users to archive or delete documents.
-

5. Non-Functional Requirements

5.1 Security

- **NFR-01:** User passwords shall be securely hashed.
- **NFR-02:** The system shall use role-based access control.
- **NFR-03:** Users shall only access data and functionalities permitted by their roles and permissions.
- **NFR-04:** All communication between the client and the API shall use encrypted HTTPS connections.
- **NFR-05:** Authentication sessions shall be securely managed.
- **NFR-06:** The system shall validate and sanitize user input.

- **NFR-07:** The system shall protect authentication endpoints against repeated unauthorized attempts.
 - **NFR-08:** Sensitive administrative actions shall be recorded in audit logs.
 - **NFR-09:** Restricted documents shall only be accessible to authorized users.
 - **NFR-10:** Contribution information shall only be accessible to authorized users.
-

5.2 Performance

- **NFR-11:** Common system operations should provide responses within an acceptable time under normal operating conditions.
 - **NFR-12:** Large data lists shall use pagination.
 - **NFR-13:** Search and filtering operations shall be optimized.
 - **NFR-14:** Frequently accessed database fields shall be indexed when necessary.
 - **NFR-15:** The system shall support multiple simultaneous users.
-

5.3 Scalability and Extensibility

- **NFR-16:** The platform shall use a modular architecture.
 - **NFR-17:** New modules shall be implemented without requiring major changes to existing modules.
 - **NFR-18:** The system shall support future functionality extensions.
 - **NFR-19:** The application shall maintain clear boundaries between modules.
 - **NFR-20:** The codebase shall allow independent customization for different church deployments.
-

5.4 Usability

- **NFR-21:** Portuguese (Brazil) shall be the default system language.
- **NFR-22:** The interface shall be responsive on desktop, tablet, and mobile devices.
- **NFR-23:** Navigation shall be consistent across all modules.
- **NFR-24:** Forms shall provide clear validation messages in Portuguese.
- **NFR-25:** The interface shall be understandable to users with different levels of technical experience.
- **NFR-26:** The application shall support future language expansion through an internationalization structure.

5.5 Reliability

- **NFR-27:** The system shall maintain database consistency.
- **NFR-28:** Important data shall be backed up regularly.
- **NFR-29:** The system shall provide clear feedback when operations fail.
- **NFR-30:** Error messages shall not expose sensitive technical information.
- **NFR-31:** The system shall preserve historical information when required.

5.6 Maintainability

- **NFR-32:** The application shall use a modular and organized code structure.
- **NFR-33:** The code shall follow consistent naming and formatting standards.
- **NFR-34:** The API shall be documented.
- **NFR-35:** Important business rules shall be covered by automated tests.
- **NFR-36:** Database changes shall be managed through migrations.
- **NFR-37:** Important architectural and deployment decisions shall be documented.

6. Complete Project Development Plan

Phase 1 — Requirements and Planning

Activities

- Review the project scope
- Validate the selected modules
- Review functional requirements
- Review non-functional requirements
- Define user roles
- Define permissions
- Define business rules
- Define module dependencies
- Prioritize the first version

Deliverables

- Requirements specification
 - User-role definition
 - Permission matrix
 - Initial product backlog
-

Phase 2 — Backend and Database Design

The backend should be developed before the frontend.

Activities

- Define database entities
- Define entity relationships
- Create the entity-relationship diagram
- Define database constraints
- Define database indexes
- Create the Prisma schema
- Define API resources
- Define API routes
- Define request and response structures
- Define authentication and authorization rules

Deliverables

- Entity-relationship diagram
 - Database schema
 - Prisma models
 - API specification
 - Backend architecture
-

Phase 3 — Backend Project Setup

Activities

- Create the Node.js project
- Configure TypeScript
- Configure Express.js
- Configure PostgreSQL

- Configure Prisma ORM
- Configure environment variables
- Configure database migrations
- Configure code formatting
- Configure linting
- Configure testing tools
- Configure centralized error handling
- Configure application logging

Deliverables

- Initial backend application
 - Database connection
 - Prisma configuration
 - Development environment
-

Phase 4 — Authentication and Authorization

Activities

- Implement user registration
- Implement login
- Implement logout
- Implement password hashing
- Implement secure session management
- Implement password reset
- Implement account activation and deactivation
- Implement roles
- Implement permissions
- Implement authorization middleware
- Implement protected API routes

Deliverables

- Authentication API
 - Authorization system
 - Role and permission management
-

Phase 5 — Core Backend Modules

Develop the backend modules in the following order:

1. Church Management
2. Member Management
3. Ministry Management
4. Announcement Management
5. Event Management
6. Inventory Management
7. Contribution Management
8. Bible Studies Management
9. Document Management

For each module:

1. Define the database entities.
2. Create or update Prisma models.
3. Create database migrations.
4. Create validation schemas.
5. Implement business rules.
6. Create services.
7. Create API controllers.
8. Create API routes.
9. Apply authentication.
10. Apply authorization.
11. Write automated tests.
12. Document the API.

Phase 6 — API Testing and Validation

Activities

- Test API endpoints
- Test authentication
- Test permissions
- Test data validation
- Test business rules
- Test contribution limits
- Test announcement destinations
- Test database constraints

- Test error responses

Deliverables

- Tested backend API
 - API test report
 - Corrected backend version
-

Phase 7 — Frontend Development

The frontend should begin after the core backend API is stable.

Activities

- Create the Next.js application
- Configure TypeScript
- Configure Portuguese as the default language
- Configure translation files
- Create the general application layout
- Create navigation
- Create authentication pages
- Create protected routes
- Create reusable components
- Connect the frontend to the API

Main Pages

- Login
- Password recovery
- Dashboard
- Church settings
- User management
- Member list
- Member profile
- Ministry management
- Announcement management
- Event calendar
- Inventory management
- Contribution management
- Bible studies
- Document management

Phase 8 — Frontend Module Implementation

Implement the frontend modules in the same order as the backend:

1. Church Management
2. Member Management
3. Ministry Management
4. Announcement Management
5. Event Management
6. Inventory Management
7. Contribution Management
8. Bible Studies Management
9. Document Management

For each module:

- Create list pages
- Create registration forms
- Create detail pages
- Create editing interfaces
- Implement search and filters
- Display validation messages in Portuguese
- Apply permission-based interface controls
- Connect the interface to the API
- Test responsive behavior

Phase 9 — Integration and System Testing

Activities

- Test frontend and backend integration
- Test complete user flows
- Test role-based permissions
- Test announcement destinations
- Test event participation
- Test financial contributions
- Test item-based contributions
- Test contribution limits
- Test document permissions

- Test responsive behavior
- Test error handling

Deliverables

- Integrated platform
 - System test report
 - Corrected version
-

Phase 10 — User Validation

Activities

- Select church administrators and users
- Demonstrate the system
- Allow users to perform representative tasks
- Collect usability feedback
- Identify missing requirements
- Correct usability issues

Deliverables

- User feedback report
 - Requirement adjustments
 - Improved system version
-

Phase 11 — Deployment

Activities

- Deploy the backend API
- Deploy the frontend
- Deploy the PostgreSQL database
- Configure HTTPS
- Configure environment variables
- Configure file storage
- Configure database backups
- Configure monitoring
- Configure error tracking

Deliverables

- Production platform
 - Deployment documentation
 - Backup configuration
 - Monitoring configuration
-

7. Authentication and Authorization Plan

7.1 Centralized Authentication

Authentication will be implemented as a shared system service.

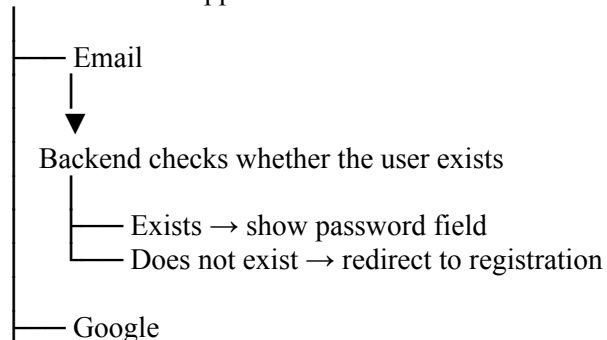
All modules will use the same authentication and authorization infrastructure.

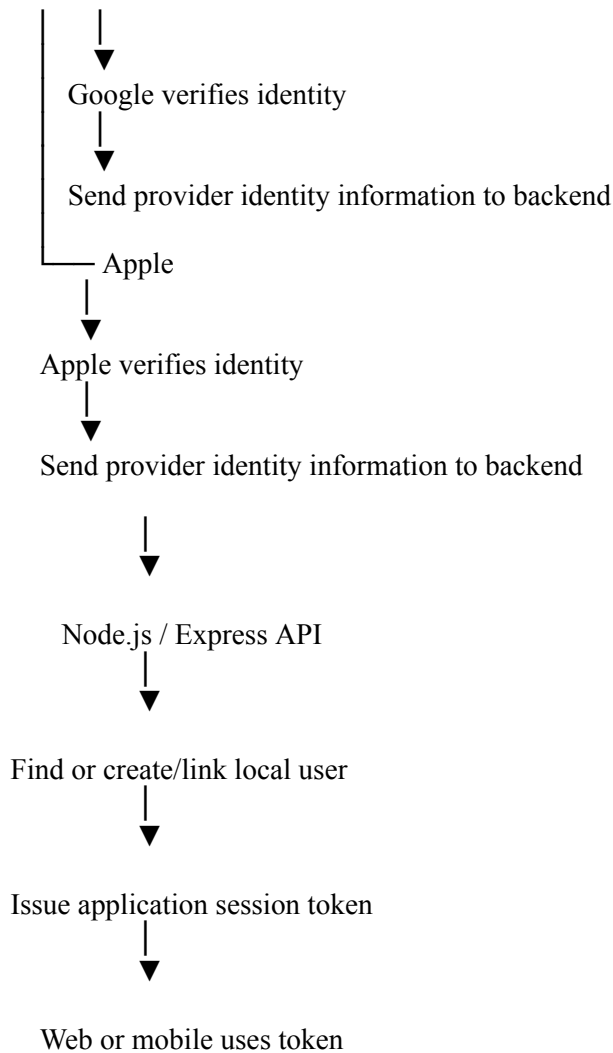
Authentication should provide:

- User registration
 - Login
 - Logout
 - Password recovery
 - Password reset
 - Password change
 - Account activation
 - Account deactivation
 - Session management
-

7.2 Authentication Flow

Web application / Mobile application

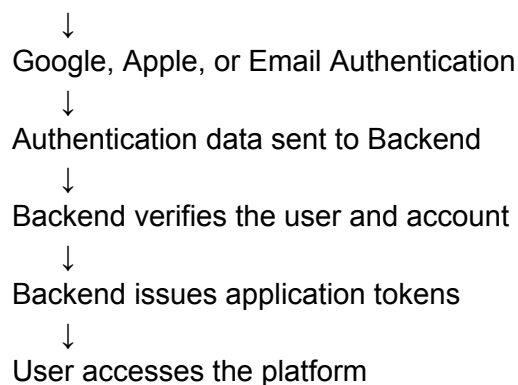




a. General Architecture

- **Web:** Auth.js handles Google and Apple authentication.
- **Mobile:** The mobile application uses the appropriate Google and Apple authentication providers.
- **Backend:** The Node.js/Express API verifies the authentication data, identifies or links the user, checks the account status, and issues the platform's authentication tokens.
- **Database:** Stores users, provider accounts, roles, approval status, and sessions.

Web / Mobile



b. Email and Password Flow

1. The user enters their email address.
2. The frontend sends the email to the backend.
3. The backend checks whether the email is registered.
4. If the email exists:
 - The frontend displays the password field.
 - The user enters their password.
 - The backend validates the password.
5. If the email does not exist:
 - The user is redirected to the registration page.
6. After successful authentication, the backend checks the user's approval status.
7. If the account is approved, the backend issues the authentication tokens.
8. If the account is pending approval, access is denied and the user is informed that their registration is awaiting administrator approval.

Security measures:

- Rate limiting on authentication routes.
- CAPTCHA triggered after suspicious or repeated authentication attempts.
- Passwords stored using a secure hashing algorithm.
- Generic error messages for invalid login attempts.

c. Google and Apple Authentication Flow

1. The user selects **Continue with Google** or **Continue with Apple**.
2. Authentication is performed by the provider on the web or mobile application.
3. After successful authentication, the client receives the provider authentication credential.
4. The client sends the credential and provider information to the backend.
5. The backend verifies the provider credential and account ID.
6. The backend searches for an existing linked provider account.
7. If the provider account is already linked:
 - The associated user is identified.
8. If the provider account is not linked:
 - The backend checks for an existing user with the verified email.
 - If a valid matching account exists, the provider account may be linked according to the account-linking rules.
 - If no user exists, the user is redirected to complete registration.
9. The backend checks the user's approval status.
10. If the account is approved, the backend issues the platform authentication tokens.

d. Registration and Church Approval Flow

1. A new user completes the registration form.
2. The backend creates the user with a status such as:

PENDING_APPROVAL

3. The user's registration is visible to church administrators.
4. An administrator reviews the registration.
5. The administrator can:

APPROVE

REJECT

6. If approved:
 - The user's status changes to:

ACTIVE

- The user can access the church platform.
7. If the registration is pending or rejected:
 - The backend does not issue an active platform session.
 - The user receives the appropriate status message.

e. Suggested User Statuses

PENDING_APPROVAL

ACTIVE

REJECTED

SUSPENDED

INACTIVE

Their meaning would be:

- **PENDING_APPROVAL:** Registered but waiting for administrator approval.
- **ACTIVE:** Approved and allowed to access the platform.
- **REJECTED:** Registration was not approved.
- **SUSPENDED:** Access was temporarily blocked.
- **INACTIVE:** Account is no longer active.

d. Account Linking

The backend remains responsible for account linking.

Provider authentication



Client receives provider credential



Client sends credential to backend



Backend verifies provider identity

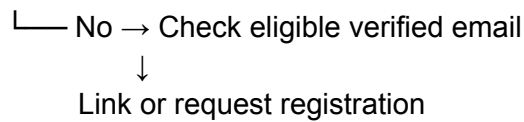


Backend checks provider account ID



Existing provider account?

└─ Yes → Identify linked user



The backend will not trust only the email or user information received from the frontend. It will verify the provider credential and provider account ID before granting access.

g. Token and Session Flow

After successful authentication and approval:

User authenticated



Account status is ACTIVE



Backend creates a session



Backend issues:

- Access token
- Refresh token



User accesses protected resources

- The **access token** is used for authenticated API requests.
- The **refresh token** is used to obtain a new access token.
- The backend validates the user and permissions on every protected request.
- The frontend uses authentication information to control the interface, but the backend remains responsible for enforcing access permissions.

h. Final Authentication Rule

A user can access the church platform only when all the following conditions are met:

Identity successfully authenticated

+

Provider credential verified by backend

+

User account exists

+

Church administrator approved the account

+

User account status is ACTIVE

=

Platform access granted

This provides a simple foundation for implementation while keeping the authentication system compatible with the current web application and the future mobile application.

7.4 Role-Based Access Control

The system should use **Role-Based Access Control (RBAC)**.

Initial roles may include:

Role	Main Access
System Administrator	Full access to the platform
Church Administrator	Access to church management and administrative modules
Church Leader	Access to selected church information and management features
Ministry Leader	Access to assigned ministries and related information
Member	Access to personal information, announcements, events, and permitted features

The final roles should be validated with the church because organizational structures may differ.

7.5 Authorization Across Modules

Authorization must be enforced by the backend.

Examples:

- Only authorized administrators may manage church settings.
- Only authorized users may register or edit members.
- Ministry leaders may manage information related to their assigned ministries.
- Users may only view announcements directed to them.
- Only authorized users may manage contribution campaigns.
- Only authorized users may access contribution records.
- Users may only access documents permitted by their role or group.

The frontend should hide or disable unauthorized actions, but the backend must always perform the final permission verification.

8. Recommended Technical Architecture

Frontend

- Next.js
- TypeScript
- Responsive interface
- Reusable components
- Form validation
- Protected routes
- Portuguese as the default language
- Internationalization-ready structure

Backend

- Node.js
- Express.js
- TypeScript
- REST API
- Modular architecture
- Centralized error handling
- Request validation
- Authentication middleware
- Authorization middleware

Database

- PostgreSQL

- Prisma ORM
- Database migrations
- Foreign-key constraints
- Database indexes
- Transaction support for critical operations

File Storage

Use object storage for:

- Church logos
- Announcement images
- Bible study materials
- Church documents

Store file metadata and file references in PostgreSQL rather than storing large files directly in the database.

9. Suggested Backend Structure

```
src/  
├── config/  
├── common/  
│   ├── errors/  
│   ├── middleware/  
│   ├── validation/  
│   ├── logging/  
│   └── utilities/  
├── modules/  
│   ├── authentication/  
│   ├── users/  
│   ├── roles/  
│   ├── church/  
│   ├── members/  
│   ├── ministries/  
│   ├── announcements/  
│   ├── events/  
│   ├── inventory/  
│   ├── contributions/  
│   ├── bible-studies/  
│   └── documents/  
├── routes/  
└── app.ts
```

└─ server.ts

Each module should contain its own:

- Routes
 - Controllers
 - Services
 - Repositories
 - Validation schemas
 - Database access logic
 - Business rules
 - Tests
-

10. Additional Necessary Plans

10.1 Database Plan

- Define entities before implementation.
- Create an entity-relationship diagram.
- Use Prisma migrations.
- Apply foreign-key constraints.
- Add indexes to frequently searched fields.
- Use transactions for critical operations.
- Use soft deletion when historical information must be preserved.
- Include `createdAt` and `updatedAt` fields in important entities.

The database does not need multi-church identifiers because each system installation will use a separate database.

10.2 Contribution Consistency Plan

Financial and item-based contributions require additional data consistency controls.

For item-based contributions:

- Validate available quantity before creating a commitment.
- Validate the maximum number of contributors.
- Prevent the required quantity from being exceeded.
- Use database transactions when multiple users may attempt to commit to the same item simultaneously.
- Recalculate availability after a commitment is created, updated, or canceled.

These controls are important to prevent two users from simultaneously claiming quantities that exceed the established limit.

10.3 Announcement Destination Plan

Announcements should use a flexible destination structure.

Possible destinations:

- Members
- Visitors
- Church leaders
- One ministry
- Multiple ministries
- All users

The backend should determine whether a user is allowed to view an announcement.

The frontend should only display announcements returned by the authorized API request.

10.4 Language and Internationalization Plan

Portuguese (Brazil) will be the default language.

The system should:

- Store interface text in translation files.
- Use Portuguese validation messages.
- Use Portuguese notification messages.
- Support future language additions without major code changes.
- Avoid permanently embedding interface text throughout application logic.

10.5 Validation Plan

Validate information at multiple levels:

- Frontend form validation
- Backend request validation
- Business-rule validation
- Database constraints

The backend must validate all data, even when the frontend already performs validation.

10.6 File Management Plan

- Define permitted file types.
 - Define maximum file sizes.
 - Validate uploaded files.
 - Store files in object storage.
 - Store file metadata in the database.
 - Restrict access to private documents.
 - Allow authorized users to download files.
-

10.7 Notification Plan

Initial version:

- In-app notifications

Possible future additions:

- Email notifications
- Mobile push notifications
- Messaging integrations

Potential notification events:

- New announcements
- Event reminders

- Contribution deadlines
 - Contribution updates
 - Bible study reminders
 - Document updates
-

10.8 Audit Plan

Record important administrative actions, including:

- User creation
- Role changes
- Member registration and updates
- Ministry leadership changes
- Announcement publication
- Event changes
- Inventory movements
- Contribution changes
- Document uploads and deletions

Audit records should include:

- User responsible
 - Action performed
 - Date and time
 - Affected record
-

10.9 Testing Plan

Use several testing levels:

- Unit tests for business logic
- Integration tests for API endpoints
- End-to-end tests for important user flows
- Manual usability testing

Priority flows:

- User login
- Church configuration

- Member registration
 - Ministry assignment
 - Announcement publication
 - Announcement destination filtering
 - Event registration
 - Financial contribution registration
 - Item contribution commitment
 - Contribution-limit validation
 - Document upload
 - Document permission validation
-

10.10 Backup and Recovery Plan

- Perform regular database backups.
 - Store backups securely.
 - Define a recovery procedure.
 - Test database restoration periodically.
 - Protect uploaded files.
 - Maintain recoverable copies of important documents.
-

10.11 Documentation Plan

Maintain:

- Project overview
 - Installation guide
 - Environment configuration guide
 - API documentation
 - Database documentation
 - Architecture documentation
 - Deployment guide
 - User guide in Portuguese
 - Administrator guide in Portuguese
-

11. Recommended Development Order

Foundation

1. Requirements review
2. Database design
3. Backend project setup
4. PostgreSQL configuration
5. Prisma configuration
6. Authentication
7. Authorization
8. User and role management
9. Church configuration

Core Modules

10. Member Management
11. Ministry Management
12. Announcement Management
13. Event Management

Operational Modules

14. Inventory Management
15. Contribution Management
16. Bible Studies Management
17. Document Management

Frontend

18. Next.js project setup
19. Portuguese interface configuration
20. Authentication interface
21. Dashboard
22. Module interfaces
23. API integration
24. Responsive implementation

Finalization

25. Integration testing

26. Security review
 27. Performance review
 28. Church-user validation
 29. Corrections
 30. Deployment
 31. Documentation
-

12. Initial Minimum Viable Product

The first usable version should contain:

- Backend API
- Database
- Authentication
- Authorization
- User roles
- Church configuration
- Member Management
- Ministry Management
- Announcement Management
- Basic Event Management
- Responsive Portuguese interface

The remaining modules can be developed and released incrementally.

This approach allows the church to begin using the core system while more specialized modules are implemented and validated.